

DataPrep AI Copilot

Liaison Backend / Frontend

Contrat API, flux de données & intégration technique

Version 1.0 — Avril 2026

1. Vue d'ensemble

Ce document décrit comment le frontend React communique avec le backend FastAPI. Il couvre la configuration de base, le flux complet par page, et le détail de chaque endpoint : méthode HTTP, payload envoyé, structure de réponse, et comment exploiter les données côté interface.

Principe fondamental : le frontend ne contient aucune logique métier. Il envoie des actions, reçoit des résultats et les affiche. Toute la logique de décision (analyse, diagnostic, LLM) est côté backend.

2. Configuration de base

2.1 Client Axios

Toutes les requêtes passent par une instance Axios centrale. Le header X-Session-UUID est injecté automatiquement sur chaque requête à partir du store Zustand.

```
// src/api/client.js
import axios from 'axios';
import { useSessionStore } from '../store/sessionStore';

const client = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 60000, // 60s - les appels LLM peuvent être longs
});

client.interceptors.request.use((config) => {
  const uuid = useSessionStore.getState().sessionUUID;
  if (uuid) config.headers['X-Session-UUID'] = uuid;
  return config;
});

export default client;
```

2.2 Session UUID

Au premier chargement de l'app, le frontend vérifie localStorage. S'il n'y a pas de UUID valide, il en crée un via l'API et le persiste.

```
// src/store/sessionStore.js (init au démarrage App.jsx)
const stored = localStorage.getItem('session_uuid');
if (stored) {
  // Vérifier que la session est encore valide
  GET /api/sessions/{stored}
  // → Si valide : utiliser le UUID existant
  // → Si expiré ou 404 : en créer un nouveau
} else {
```

```

POST /api/sessions
// → Stocker le UUID retourné dans localStorage + Zustand
}

```

3. Flux par page

3.1 Page Home — Upload

Étape	Appel API	Payload	Réponse exploitée
1. Upload	POST /api/datasets/upload	FormData : file, target_column, task_type, objective	dataset.id → stocker dans Zustand
2. Analyse	POST /api/analyze/{id}	— (dataset_id dans l'URL)	eda, diagnostic, ordered_problems, justification
3. Redirect	— (navigation React)	—	Redirection vers /eda avec les données en store

L'appel /api/analyze est le seul appel lourd du démarrage. Il retourne EDA + Diagnostic + Ordre LLM en une seule requête. Ne pas appeler /eda et /diagnostic séparément au chargement initial.

3.2 Page EDA — Visualisation

Les données EDA sont déjà disponibles dans le store Zustand après l'appel /analyze. Pas de nouvel appel API nécessaire au montage de la page.

Mapping données → graphiques

Champ EDA reçu	Composant React	Graphique
col.numeric.histogram	HistogramChart	BarChart Recharts (bins/fréquence)
col.numeric.q25, q75, min, max	BoxPlot	SVG custom ou composant dédié
col.categorical.top_values	CategoryBarChart	BarChart horizontal
eda.correlation_matrix	CorrelationHeatmap	Grille colorée custom
eda.target_correlations	TargetCorrelBar	BarChart horizontal trié
eda.top_correlation_pairs	ScatterSuggestion	Suggestions de scatter plots
eda.missing_combinations	MissingPatternTable	Tableau avec badges colonnes

Champ EDA reçu	Composant React	Graphique
eda.target_profile.class_pct	ClassDistPie	PieChart déséquilibre classes

3.3 Page Preprocessing — Flux de résolution

C'est la page la plus interactive. Chaque action déclenche un appel API dans un ordre précis.

Récupérer les recommandations

```
// Appel déclenché quand l'utilisateur sélectionne un problème
POST /api/recommend/{dataset_id}
Body: { "problem_index": 2 } // index 0-based dans la liste

// Réponse
{
  "solutions": [
    {
      "id": "imputation_mediane",
      "nom": "Imputation par la médiane",
      "explication": "...",
      "effets_secondaires": "...",
      "requires_code": false,
      "transformations": [
        { "function": "impute_median", "params": { "column": "Age" } }
      ],
      "code": null
    }
  ],
  "recommandation": { "id": "imputation_mediane", "justification": "..." }
}
```

Appliquer une transformation

```
// Après que l'utilisateur a cliqué Appliquer
POST /api/execute/{dataset_id}
Body: { "function": "impute_median", "params": { "column": "Age" } }

// Si la solution nécessite plusieurs transformations, les envoyer en séquence
// (chaque réponse retourne le dataset_id mis à jour)

// Réponse
{
  "dataset_id": "...",
  "rows_before": 891, "rows_after": 891,
  "cols_before": 12, "cols_after": 12,
  "eda": { ... }, // EDA re-calculé sur le dataset transformé
  "diagnostic": { ... } // Nouveaux problèmes détectés
}
```

Exécuter du code LLM personnalisé

```
// Étape 1 : générer le code
POST /api/recommend/{dataset_id}/generate-code
Body: { "problem_index": 2, "user_intent": "Je veux grouper par Sex avant
d'imputer" }

// Réponse
{
  "code": "df['Age'] = df.groupby('Sex')['Age'].transform(lambda x:
x.fillna(x.median()))",
  "description": "Imputation de l'âge par la médiane du groupe selon le sexe",
  "warning": null,
  "static_warnings": []
}

// Étape 2 : afficher le code dans CodeViewer pour validation utilisateur
// Étape 3 : exécuter le code validé
POST /api/execute/{dataset_id}/code
Body: { "code": "df['Age'] = df.groupby..." }
```

3.4 Page Export — Download

```
// Aperçu du dataset (tableau dans l'interface)
GET /api/datasets/{dataset_id}/preview?n=10
// → { columns, dtypes, rows, total_rows }

// Téléchargement version transformée
GET /api/datasets/{dataset_id}/download?version=latest
// → Fichier CSV/Excel en streaming (Content-Disposition: attachment)

// Téléchargement version originale brute
GET /api/datasets/{dataset_id}/download?version=raw
```

4. Contrat API complet

Méthode	Endpoint	Page déclencheuse	Utilisation
POST	/api/sessions	App init	Créer une session UUID
GET	/api/sessions/{uuid}	App init	Vérifier session existante
POST	/api/datasets/upload	Home	Upload + enregistrement dataset
GET	/api/datasets/{id}	Toutes	Métadonnées du dataset actif
GET	/api/datasets/{id}/preview	Export	Aperçu des N premières lignes
GET	/api/datasets/{id}/download	Export	Télécharger raw ou transformé

Méthode	Endpoint	Page déclencheuse	Utilisation
PATCH	/api/datasets/{id}/target	Home	Modifier la cible après upload
DELETE	/api/datasets/{id}	Home	Supprimer le dataset
POST	/api/analyze/{id}	Home → EDA	EDA + Diagnostic + Ordre LLM
POST	/api/eda/{id}	EDA (optionnel)	EDA seul si besoin de refresh
POST	/api/diagnostic/{id}	Preprocessing	Diagnostic seul si besoin
POST	/api/recommend/{id}	Preprocessing	Solutions LLM pour un problème
POST	/api/recommend/{id}/generate-code	Preprocessing	Génération code Python LLM
POST	/api/execute/{id}	Preprocessing	Appliquer une transformation
POST	/api/execute/{id}/code	Preprocessing	Exécuter code Python LLM
GET	/api/messages/{session_uuid}	Toutes	Historique des échanges LLM
POST	/api/messages	Preprocessing	Sauvegarder un échange LLM

5. Gestion des erreurs frontend

Code HTTP	Situation	Comportement UI recommandé
400	Payload invalide (ex: colonne inexistante)	Toast d'erreur inline sous le champ concerné
404	Dataset introuvable (session expirée)	Redirection Home + message de session expirée
413	Fichier trop volumineux	Message sous la zone d'upload avec la limite
500	Erreur serveur inattendue	Toast d'erreur générique avec bouton Réessayer
503	LLM indisponible (rate limit)	Message spécifique avec countdown de réessai
Timeout (60s)	Appel LLM trop long	Toast avec option d'annulation et réessai

6. Stratégie de cache (TanStack Query)

TanStack Query gère automatiquement les loading states, le cache et le refetch. Voici les query keys recommandées pour cohérence entre les composants.

```
// Clés de query recommandées
['session', uuid] // Session check
['dataset', datasetId] // Métadonnées dataset
['dataset', datasetId, 'preview'] // Aperçu lignes
['analysis', datasetId] // EDA + Diagnostic (résultat /analyze)
['recommend', datasetId, problemIndex] // Solutions LLM par problème

// Invalidation après exécution d'une transformation
// → Invalider ['analysis', datasetId] pour forcer le refresh
queryClient.invalidateQueries({ queryKey: ['analysis', datasetId] });

// Stale time recommandé
staleTime: 5 * 60 * 1000 // 5 min pour EDA (re-calculé seulement si
transformation)
staleTime: 0 // 0 pour les recommandations LLM (toujours frais)
```

7. Variables d'environnement frontend

```
# .env (développement local)
VITE_API_BASE_URL=http://localhost:8000
VITE_SESSION_EXPIRY_DAYS=7

# .env.production (Netlify)
VITE_API_BASE_URL=https://ton-backend.railway.app
VITE_SESSION_EXPIRY_DAYS=7
```

Sur Netlify, configurer VITE_API_BASE_URL dans les variables d'environnement du site (Settings → Environment variables). Ne jamais commiter le .env de production dans Git.